在提供适当署名的前提下，Google 特此授予仅为新闻或学术作品复制本文中表格和图形的许可。

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

主流的序列转导模型基于包含编码器和解码器的复杂循环或卷积神经网络。性能最佳的模型还通过注意力机制连接编码器和解码器。我们提出了一种新的简单网络架构——Transformer，它完全基于注意力机制，完全摒弃了循环和卷积。在两个机器翻译任务上的实验表明，这些模型在质量上更优，同时更具可并行性，并且需要的训练时间显著减少。我们的模型在 WMT 2014 英语-德语翻译任务上取得了 28.4 的 BLEU 分数，比现有最佳结果（包括集成模型）提高了超过 2 个 BLEU 分数。在 WMT 2014 英语-法语翻译任务上，我们的模型在 8 个 GPU 上训练 3.5 天后，建立了新的单模型最先进 BLEU 分数 41.8，这仅是文献中最佳模型训练成本的一小部分。我们通过将 Transformer 成功应用于英语成分句法分析（无论训练数据量大小），表明其能很好地推广到其他任务。

1 引言

循环神经网络，尤其是长短期记忆网络 [13] 和门控循环单元网络 [7]，已被牢固确立为序列建模和转换问题（如语言建模和机器翻译）中最先进的方法 [35, 2, 5]。此后，许多努力不断推动循环语言模型和编码器-解码器架构的边界 [38, 24, 15]。

循环模型通常沿着输入和输出序列的符号位置分解计算。将位置与计算时间步骤对齐，它们生成一系列隐藏状态 h_t，作为前一个隐藏状态 h_{t-1} 和位置 t 的输入的函数。这种固有的顺

*贡献相等。作者列表顺序随机。Jakob 提出用自注意力机制替换 RNN 并开始评估这一想法。Ashish 与 Illia 设计并实现了第一个 Transformer 模型，并参与了这项工作的各个方面。Noam 提出了缩放点积注意力、多头注意力以及无参数的位置表示，并成为几乎参与每个细节的另一位成员。Niki 在我们原始的代码库和 tensor2tensor 中设计、实现、调整和评估了无数模型变体。Llion 也尝试了新颖的模型变体，负责我们最初的代码库、高效推理和可视化。Lukasz 和 Aidan 花费了无数个漫长的日子设计各个部分并实现 tensor2tensor，取代了我们早期的代码库，极大地改善了结果并加速了我们的研究。

†工作完成于 Google Brain 期间。

‡工作完成于 Google Research 期间。

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

序性质阻止了训练示例内的并行化，这在序列长度较长时变得关键，因为内存限制限制了示例间的批处理。

近期工作通过分解技巧 [21] 和条件计算 [32] 实现了计算效率的显著提升，同时在后者的情况下还提高了模型性能。然而，顺序计算的基本约束仍然存在。

注意力机制已成为各种任务中引人注目的序列建模和转换模型不可或缺的组成部分，允许对依赖关系进行建模而无视其在输入或输出序列中的距离 [2, 19]。但在除少数情况外 [27]，此类注意力机制与循环网络结合使用。

在这项工作中，我们提出了 Transformer，一种避免循环并完全依赖注意力机制来绘制输入和输出之间全局依赖关系的模型架构。Transformer 允许显著更多的并行化，并且在八个 P100 GPU 上训练仅十二小时后即可达到翻译质量的新 state of the art。

2 背景

减少序列计算的目标同样构成了扩展神经 GPU[16]、ByteNet[18] 和 ConvS2S[9] 的基础，这些模型均使用卷积神经网络作为基本构建模块，并行计算所有输入和输出位置的隐藏表示。在这些模型中，关联两个任意输入或输出位置信号所需的操作次数随位置间距离的增长而增加：ConvS2S 为线性增长，ByteNet 为对数增长。这使得学习远距离位置之间的依赖关系变得更加困难 [12]。在 Transformer 中，这一操作次数被减少到一个常数级别，尽管这是以降低有效分辨率为代价的（因为对注意力加权的位置进行了平均），我们通过第 3.2节所述的多头注意力机制来抵消这种影响。

自注意力（Self-attention），有时也称为内部注意力（intra-attention），是一种关联单个序列中不同位置的注意力机制，目的是计算该序列的表示。自注意力已成功应用于多种任务，包括阅读理解、抽象摘要、文本蕴含学习以及任务无关的句子表示学习 [4, 27, 28, 22]。

端到端记忆网络基于循环注意力机制而非序列对齐的循环，并已被证明在简单语言问答和语言建模任务上表现良好 [34]。

然而，据我们所知，Transformer 是第一个完全依赖自注意力来计算其输入和输出表示的转换模型，而没有使用序列对齐的 RNN 或卷积。在接下来的章节中，我们将描述 Transformer 的架构，解释自注意力的动机，并讨论其相对于 [17, 18] 和 [9] 等模型的优势。

3 模型架构

大多数具有竞争力的神经序列转导模型都具有编码器-解码器结构 [5, 2, 35]。这里，编码器将符号表示的输入序列 $(x_1, ..., x_n)$ 映射到连续表示的序列 $\mathbf{z} = (z_1, ..., z_n)$。给定 $\mathbf{z}$，解码器然后一次一个元素地生成符号的输出序列 $(y_1, ..., y_m)$。在每一步，模型是自回归的 [10]，在生成下一个符号时消耗先前生成的符号作为额外输入。

Transformer 遵循这种整体架构，对编码器和解码器使用堆叠的自注意力和逐点全连接层，分别如图 1 的左右半部分所示。

3.1 编码器和解码器堆叠

编码器： 编码器由 $N = 6$ 个相同层堆叠而成。每层有两个子层。第一个是多头自注意力机制，第二个是简单的逐位置全连接前馈网络。我们在每个子层周围使用残差连接 [11]，然后进行层归一化 [1]。也就是说，每个子层的输出是 $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$，其中 $\mathrm{Sublayer}(x)$ 是子层本身实现的函数。为了促进这些残差连接，模型中的所有子层以及嵌入层都产生维度 $d_{\mathrm{model}} = 512$ 的输出。

解码器： 解码器也由 $N = 6$ 个相同层堆叠而成。除了每个编码器层中的两个子层外，解码器插入第三个子层，该子层对编码器堆叠的输出执行多头注意力。与编码器类似，我们在每个子层周围使用残差连接，然后进行层归一化。我们还修改了解码器堆叠中的自注意力子层，以防止位置关注到后续位置。这种掩码结合输出嵌入偏移一个位置的事实，确保对位置 i 的预测只能依赖于位置小于 i 的已知输出。

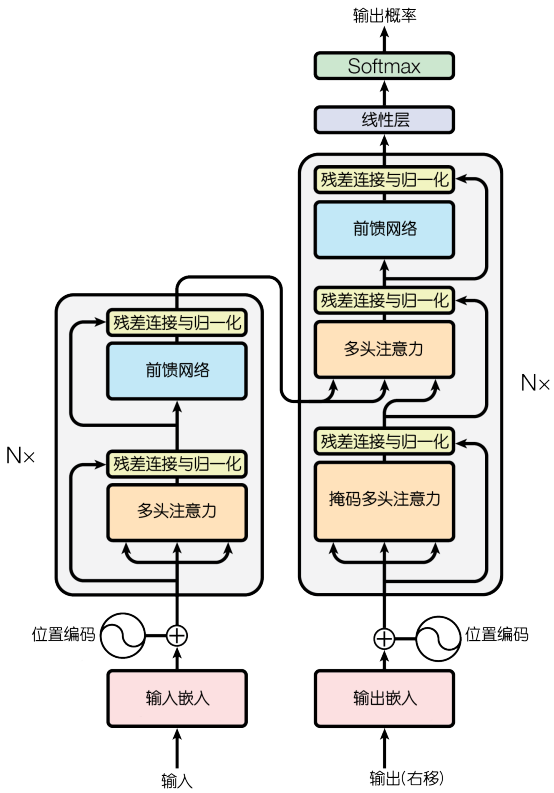

Figure 1: Transformer 模型架构。

3.2 注意力

注意力函数可以描述为将查询和一组键值对映射到输出，其中查询、键、值和输出都是向量。输出计算为值的加权和，其中分配给每个值的权重通过查询与相应键的兼容性函数计算。

3.2.1 缩放点积注意力

我们将我们的特定注意力称为“缩放点积注意力”（图 2）。输入由维度 d_k 的查询和键，以及维度 d_v 的值组成。我们计算查询与所有键的点积，每个除以 $\sqrt{d_k}$，然后应用 softmax 函数来获得值的权重。

在实践中，我们同时计算一组查询的注意力函数，将它们打包成矩阵 Q。键和值也打包成矩阵 K 和 V。我们计算输出矩阵为：

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{1}$$

两种最常用的注意力函数是加性注意力 [2] 和点积（乘法）注意力。点积注意力与我们的算法相同，除了缩放因子 $\frac{1}{\sqrt{d_k}}$。加性注意力使用具有单个隐藏层的前馈网络计算兼容性函数。虽然两者在理论复杂度上相似，但点积注意力在实践中更快且更节省空间，因为它可以使用高度优化的矩阵乘法代码实现。

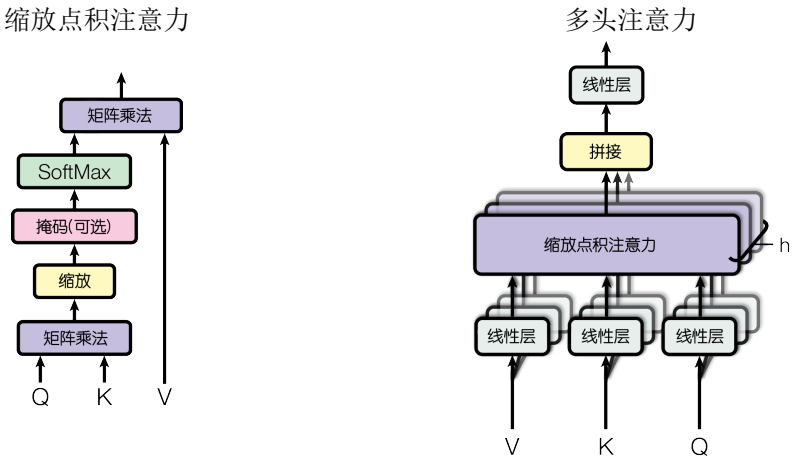

Figure 2: （左）缩放点积注意力。（右）多头注意力由几个并行运行的注意力层组成。

虽然对于较小的 d_k 值，两种机制表现相似，但对于较大的 d_k 值，不加缩放的点积注意力不如加性注意力 [3]。我们怀疑对于较大的 d_k 值，点积的幅度会变大，将 softmax 函数推入梯度极小的区域[4]。为了抵消这种效应，我们将点积缩放 $\frac{1}{\sqrt{d_k}}$。

3.2.2 多头注意力

我们发现，与使用 d_{model} 维键、值和查询执行单个注意力函数相比，将查询、键和值用不同的学习线性投影线性投影 h 次到 d_k、d_k 和 d_v 维度是有益的。然后我们在这些投影版本的查询、键和值上并行执行注意力函数，产生 d_v 维输出值。这些输出被连接并再次投影，得到最终值，如图 2 所示。

多头注意力允许模型在不同位置联合关注来自不同表示子空间的信息。使用单个注意力头时，平均会抑制这一点。

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,...,\text{head}_\text{h})W^O$$
$$\text{其中 head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

其中投影是参数矩阵 $W_i^Q \in \mathbb{R}^{d_{\text{model}}\times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}}\times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}}\times d_v}$ 和 $W^O \in \mathbb{R}^{hd_v\times d_{\text{model}}}$。

在这项工作中，我们使用 $h = 8$ 个并行注意力层或头。对于每个头，我们使用 $d_k = d_v = d_{\text{model}}/h = 64$。由于每个头的维度减少，总计算成本与全维单头注意力相似。

3.2.3 注意力在我们模型中的应用

Transformer 在三个方面使用多头注意力：

- 在“编码器-解码器注意力”层中，查询来自前一个解码器层，记忆键和值来自编码器的输出。这允许解码器中的每个位置关注输入序列中的所有位置。这模仿了序列到序列模型中的典型编码器-解码器注意力机制，例如 [38, 2, 9]。
- 编码器包含自注意力层。在自注意力层中，所有键、值和查询来自相同的地方，即编码器中前一层的输出。编码器中的每个位置可以关注编码器前一层中的所有位置。

[4]为了说明点积为何变大，假设 q 和 k 的分量是均值为 0、方差为 1 的独立随机变量。那么它们的点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为 0，方差为 d_k。

- 类似地，解码器中的自注意力层允许解码器中的每个位置关注解码器中包括该位置在内的所有位置。我们需要防止解码器中的向左信息流以保持自回归特性。我们通过在 softmax 的输入中将所有对应非法连接的值掩码（设置为 $-\infty$）来实现这一点。见图 2。

3.3 逐位置前馈网络

除了注意力子层外，编码器和解码器的每个层都包含一个全连接前馈网络，该网络分别且相同地应用于每个位置。这包括两个线性变换，中间有 ReLU 激活。

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

虽然线性变换在不同位置相同，但它们在层与层之间使用不同的参数。另一种描述方式是使用核大小为 1 的两个卷积。输入和输出的维度为 $d_{\text{model}} = 512$，内层维度为 $d_{ff} = 2048$。

3.4 嵌入和 Softmax

与其他序列转导模型类似，我们使用学习嵌入将输入标记和输出标记转换为维度 d_{model} 的向量。我们还使用通常的学习线性变换和 softmax 函数将解码器输出转换为预测的下一个标记概率。在我们的模型中，我们在两个嵌入层和 softmax 前线性变换之间共享相同的权重矩阵，类似于 [30]。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{\text{model}}}$。

3.5 位置编码

由于我们的模型不包含循环和卷积，为了使模型利用序列的顺序，我们必须注入关于序列中标记的相对或绝对位置的信息。为此，我们在编码器和解码器堆叠的底部向输入嵌入添加“位置编码”。位置编码具有与嵌入相同的维度 d_{model}，以便两者可以相加。位置编码有许多选择，学习的和固定的 [9]。

在这项工作中，我们使用不同频率的正弦和余弦函数：

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

其中 pos 是位置，i 是维度。也就是说，位置编码的每个维度对应一个正弦曲线。波长形成从 2π 到 $10000 \cdot 2\pi$ 的几何级数。我们选择这个函数是因为我们假设它允许模型轻松学习通过相对位置关注，因为对于任何固定偏移 k，PE_{pos+k} 可以表示为 PE_{pos} 的线性函数。

我们还尝试使用学习的位置嵌入 [9]，发现两个版本产生几乎相同的结果（见表 3 行 (E)）。我们选择正弦版本，因为它可能允许模型外推到训练期间遇到的序列长度更长的序列。

4 为何使用自注意力

在本节中，我们将自注意力层的各个方面与通常用于将一个可变长度的符号表示序列 $(x_1, ..., x_n)$ 映射到另一个等长序列 $(z_1, ..., z_n)$ 的循环层和卷积层进行比较，其中 $x_i, z_i \in \mathbb{R}^d$，例如典型序列转导编码器或解码器中的隐藏层。为了说明我们使用自注意力的动机，我们考虑了三个需求。

其一是每层的总计算复杂度。另一个是可以并行化的计算量，以所需顺序操作的最小数量来衡量。

第三个是网络中长程依赖之间的路径长度。学习长程依赖是许多序列转导任务中的关键挑战。影响学习这种依赖能力的一个关键因素是前向和反向信号在网络中必须遍历的路径长度。输入和输出序列中任意位置组合之间的路径越短，学习长程依赖就越容易 [12]。因此，我们也比较了由不同层类型组成的网络中任意两个输入和输出位置之间的最大路径长度。

如表 1 所述，自注意力层以恒定数量的顺序执行操作连接所有位置，而循环层需要 $O(n)$ 个顺序操作。在计算复杂度方面，当序列长度 n 小于表示维度 d 时，自注意力层比循环层更快，这在使用最先进的机器翻译模型中的句子表示时通常是这种情况，例如 word-piece

Table 1: 不同层类型的最大路径长度、每层复杂度和顺序操作的最小数量。n 是序列长度，d 是表示维度，k 是卷积的核大小，r 是受限自注意力中邻域的大小。

层类型	每层复杂度	顺序操作	最大路径长度
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

[38] 和 byte-pair [31] 表示。为了提高涉及非常长序列的任务的计算性能，自注意力可以限制为仅考虑输入序列中大小为 r 的邻域，以相应输出位置为中心。这将增加最大路径长度至 $O(n/r)$。我们计划在未来的工作中进一步研究这种方法。

一个核宽度 $k < n$ 的单个卷积层并不连接所有输入和输出位置对。要做到这一点，在连续核的情况下需要堆叠 $O(n/k)$ 个卷积层，或在膨胀卷积的情况下需要 $O(log_k(n))$ 个卷积层 [18]，从而增加网络中任意两个位置之间最长路径的长度。卷积层通常比循环层更昂贵，大约是 k 倍。然而，可分离卷积 [6] 显著降低了复杂度，降至 $O(k \cdot n \cdot d + n \cdot d^2)$。但即使 $k = n$，可分离卷积的复杂度也等于自注意力层和逐点前馈层的组合，这是我们模型中采用的方法。

作为额外好处，自注意力可以产生更可解释的模型。我们检查了模型中的注意力分布，并在附录中展示和讨论了示例。不仅各个注意力头清楚地学会了执行不同的任务，许多头还表现出与句子的句法和语义结构相关的行为。

5 训练

本节描述我们模型的训练方案。

5.1 训练数据和批处理

我们在标准的 WMT 2014 英德数据集上进行了训练，该数据集包含约 450 万个句子对。句子使用字节对编码 [3] 进行编码，该编码拥有一个约 37000 个词符的共享源-目标词汇表。对于英法翻译，我们使用了规模大得多的 WMT 2014 英法数据集，包含 3600 万个句子，并将词符分割为 32000 个词片词汇表 [38]。句子对根据近似序列长度进行批处理。每个训练批次包含一组句子对，其中约有 25000 个源词符和 25000 个目标词符。

5.2 硬件和计划

我们在一台配备 8 块 NVIDIA P100 GPU 的机器上训练我们的模型。对于我们使用全文所述超参数的基础模型，每个训练步骤耗时约 0.4 秒。我们训练基础模型总共 100,000 步（12 小时）。对于我们的大型模型（描述见表 3 最后一行），每一步耗时 1.0 秒。大型模型训练了 300,000 步（3.5 天）。

5.3 优化器

我们使用了 Adam 优化器 [20]，其中 $\beta_1 = 0.9$，$\beta_2 = 0.98$ 以及 $\epsilon = 10^{-9}$。我们在训练过程中根据以下公式调整学习率：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \tag{3}$$

这对应于在最初的 $warmup_steps$ 个训练步骤中线性增加学习率，之后则按步数的平方根的倒数比例减小。我们使用了 $warmup_steps = 4000$。

5.4 正则化

我们在训练中采用了三种正则化方法：

Table 2: Transformer 在英德和英法 newstest2014 测试上以更低的训练成本获得了比先前最先进模型更好的 BLEU 分数。

模型	BLEU		训练成本 (FLOPs)	
	英德	英法	英德	英法
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk 集成 [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL 集成 [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S 集成 [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (基础模型)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (大型)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

残差丢弃　我们将丢弃法 [33] 应用于每个子层的输出，然后才将其与子层输入相加并进行归一化。此外，我们在编码器和解码器堆栈中，对词嵌入和位置编码的和也应用了丢弃法。对于基础模型，我们使用丢弃率 $P_{drop} = 0.1$。

标签平滑　在训练期间，我们采用了值为 $\epsilon_{ls} = 0.1$ 的标签平滑技术 [36]。这会损害困惑度，因为模型学会了更加不确定，但能提高准确率和 BLEU 分数。

6 结果

6.1 机器翻译

在 WMT 2014 英德翻译任务上，大型 Transformer 模型（表 2 中的 Transformer (大型)）以超过 2.0 BLEU 的优势优于先前报告的最佳模型（包括集成模型），建立了新的最先进 BLEU 分数 28.4。此模型的配置列于表 3 的最后一行。训练在 8 块 P100 GPU 上耗时 3.5 天。即使我们的基础模型也超越了所有先前发布的模型和集成模型，且训练成本远低于任何竞争模型。

在 WMT 2014 英法翻译任务上，我们的大型模型获得了 41.0 的 BLEU 分数，优于所有先前发布的单一模型，且训练成本不到先前最先进模型的 1/4。用于英法翻译的 Transformer (大型) 模型使用的丢弃率为 $P_{drop} = 0.1$，而非 0.3。

对于基础模型，我们使用了通过对最后 5 个检查点（每 10 分钟保存一次）取平均得到的单一模型。对于大型模型，我们平均了最后 20 个检查点。我们使用了集束大小为 4 且长度惩罚 $\alpha = 0.6$ 的集束搜索 [38]。这些超参数是在开发集上实验后选择的。在推理过程中，我们将最大输出长度设置为输入长度加 50，但尽可能提前终止 [38]。

表 2 总结了我们的结果，并将我们的翻译质量和训练成本与文献中的其他模型架构进行了比较。我们通过将训练时间、使用的 GPU 数量以及每块 GPU 的持续单精度浮点性能估计值相乘来估计训练模型所用的浮点运算次数 [5]。

6.2 模型变体

为了评估 Transformer 不同组件的重要性，我们以多种方式改变基础模型，测量在英德翻译开发集 newstest2013 上性能的变化。我们使用了上一节描述的集束搜索，但没有进行检查点平均。我们在表 3 中展示了这些结果。

在表 3 的行 (A) 中，我们改变了注意力头的数量以及注意力键和值的维度，同时保持计算量不变，如第 3.2.2 节所述。虽然单头注意力比最佳设置差 0.9 BLEU，但头数过多时质量也会下降。

[5]我们分别使用了 2.8、3.7、6.0 和 9.5 TFLOPS 作为 K80、K40、M40 和 P100 的估计值。

Table 3: Transformer 架构的变体。未列出的值与基础模型相同。所有指标均在英德翻译开发集 newstest2013 上。列出的困惑度是根据我们的字节对编码计算的每词片困惑度，不应与每词困惑度直接比较。

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	训练步数	PPL (开发集)	BLEU (开发集)	参数量 $\times 10^6$
基础	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	使用可学习的位置嵌入代替正弦编码									4.92	25.7	
大型	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

Table 4: Transformer 能够很好地泛化到英语成分句法分析（结果基于 WSJ 第 23 节）

分析器	训练数据	**WSJ 23 F1**
Vinyals & Kaiser 等 (2014) [37]	仅 WSJ，判别式	88.3
Petrov 等 (2006) [29]	仅 WSJ，判别式	90.4
Zhu 等 (2013) [40]	仅 WSJ，判别式	90.4
Dyer 等 (2016) [8]	仅 WSJ，判别式	91.7
Transformer (4 层)	仅 WSJ，判别式	91.3
Zhu 等 (2013) [40]	半监督	91.3
Huang & Harper (2009) [14]	半监督	91.3
McClosky 等 (2006) [26]	半监督	92.1
Vinyals & Kaiser 等 (2014) [37]	半监督	92.1
Transformer (4 层)	半监督	92.7
Luong 等 (2015) [23]	多任务	93.0
Dyer 等 (2016) [8]	生成式	93.3

在表 3 的行 (B) 中，我们观察到减小注意力键大小 d_k 会损害模型质量。这表明确定兼容性并非易事，并且比点积更复杂的兼容性函数可能有益。我们在行 (C) 和 (D) 中进一步观察到，正如预期，更大的模型更好，并且丢弃对于避免过拟合非常有帮助。在行 (E) 中，我们用可学习的位置嵌入 [9] 替换了正弦位置编码，并观察到与基础模型几乎相同的结果。

6.3 英语成分句法分析

为了评估 Transformer 是否能泛化到其他任务，我们在英语成分句法分析上进行了实验。此任务面临特定挑战：输出受到强大的结构约束，并且明显长于输入。此外，RNN 序列到序列模型在小数据量情况下未能获得最先进的结果 [37]。

我们在 Penn Treebank [25] 的华尔街日报（WSJ）部分（约 4 万训练句子）上训练了一个 4 层 Transformer，其中 $d_{model} = 1024$。我们还在半监督设置下进行了训练，使用了来自 [37]

的更大的高置信度和 BerkeleyParser 语料库，约包含 1700 万句子。对于仅 WSJ 设置，我们使用了 16K 词符的词汇表；对于半监督设置，使用了 32K 词符的词汇表。

我们仅进行了少量实验以在 22 节开发集上选择丢弃率（包括注意力和残差，第 5.4节）、学习率和集束大小，所有其他参数与英德基础翻译模型保持一致。在推理过程中，我们将最大输出长度增加到输入长度加 300。对于仅 WSJ 和半监督设置，我们均使用集束大小 21 和 $\alpha = 0.3$。

我们在表 4 中的结果表明，尽管缺乏任务特定的调优，我们的模型表现惊人地好，结果优于除循环神经网络语法 [8] 之外的所有先前报告的模型。

与 RNN 序列到序列模型 [37] 相比，即使仅在 4 万句子的 WSJ 训练集上训练，Transformer 也优于 BerkeleyParser [29]。

7 结论

在这项工作中，我们提出了 Transformer，这是第一个完全基于注意力机制的序列转导模型，它用多头自注意力取代了编码器-解码器架构中最常用的循环层。

对于翻译任务，Transformer 的训练速度明显快于基于循环层或卷积层的架构。在 WMT 2014 英语-德语和 WMT 2014 英语-法语翻译任务上，我们都达到了新的最先进水平。在前一个任务中，我们最好的模型甚至超过了所有先前报道的集成模型。

我们对基于注意力模型的未来感到兴奋，并计划将其应用于其他任务。我们计划将 Transformer 扩展到处理文本以外的输入和输出模态的问题，并研究局部的、受限的注意力机制，以有效处理图像、音频和视频等大型输入和输出。减少生成的顺序性是我们的另一个研究目标。

我们用于训练和评估模型的代码可在　获取。

致谢　我们感谢 Nal Kalchbrenner 和 Stephan Gouws 富有成效的评论、修正和启发。

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 层归一化. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 通过联合学习对齐和翻译的神经机器翻译. *CoRR*, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. 神经机器翻译架构的大规模探索. *CoRR*, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong, and Mirella Lapata. 用于机器阅读的长短期记忆网络. *arXiv preprint arXiv:1601.06733*, 2016.

[5] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 使用 RNN 编码器-解码器学习统计机器翻译的短语表示. *CoRR*, abs/1406.1078, 2014.

[6] Francois Chollet. Xception: 使用深度可分离卷积进行深度学习. *arXiv preprint arXiv:1610.02357*, 2016.

[7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. 门控循环神经网络在序列建模上的实证评估. *CoRR*, abs/1412.3555, 2014.

[8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. 循环神经网络语法. In *Proc. of NAACL*, 2016.

[9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. 卷积序列到序列学习. *arXiv preprint arXiv:1705.03122v2*, 2017.

[10] Alex Graves. 用循环神经网络生成序列. *arXiv preprint arXiv:1308.0850*, 2013.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 图像识别的深度残差学习. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. 循环网络中的梯度流：学习长期依赖性的困难, 2001.

[13] Sepp Hochreiter and Jürgen Schmidhuber. 长短期记忆. *Neural computation*, 9(8):1735–1780, 1997.

[14] Zhongqiang Huang and Mary Harper. 跨语言使用潜在注释自训练 PCFG 语法. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.

[15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. 探索语言建模的极限. *arXiv preprint arXiv:1602.02410*, 2016.

[16] Łukasz Kaiser and Samy Bengio. 主动记忆能取代注意力吗？ In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.

[17] Łukasz Kaiser and Ilya Sutskever. 神经 GPU 学习算法. In *International Conference on Learning Representations (ICLR)*, 2016.

[18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. 线性时间的神经机器翻译. *arXiv preprint arXiv:1610.10099v2*, 2017.

[19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. 结构化注意力网络. In *International Conference on Learning Representations*, 2017.

[20] Diederik Kingma and Jimmy Ba. Adam: 一种随机优化方法. In *ICLR*, 2015.

[21] Oleksii Kuchaiev and Boris Ginsburg. LSTM 网络的分解技巧. *arXiv preprint arXiv:1703.10722*, 2017.

[22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. 一种结构化的自注意力句子嵌入. *arXiv preprint arXiv:1703.03130*, 2017.

[23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. 多任务序列到序列学习. *arXiv preprint arXiv:1511.06114*, 2015.

[24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. 基于注意力的神经机器翻译的有效方法. *arXiv preprint arXiv:1508.04025*, 2015.

[25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. 构建一个大型的英语标注语料库：宾州树库. *Computational linguistics*, 19(2):313–330, 1993.

[26] David McClosky, Eugene Charniak, and Mark Johnson. 有效的句法分析自训练. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.

[27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. 可分解注意力模型. In *Empirical Methods in Natural Language Processing*, 2016.

[28] Romain Paulus, Caiming Xiong, and Richard Socher. 一种用于抽象摘要的深度强化模型. *arXiv preprint arXiv:1705.04304*, 2017.

[29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. 学习准确、紧凑和可解释的树标注. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.

[30] Ofir Press and Lior Wolf. 使用输出嵌入改进语言模型. *arXiv preprint arXiv:1608.05859*, 2016.

[31] Rico Sennrich, Barry Haddow, and Alexandra Birch. 使用子词单元进行稀有词的神经机器翻译. *arXiv preprint arXiv:1508.07909*, 2015.

[32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 极其庞大的神经网络：稀疏门控的混合专家层. *arXiv preprint arXiv:1701.06538*, 2017.

[33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: 一种防止神经网络过拟合的简单方法. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. 端到端记忆网络. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.

[35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. 使用神经网络进行序列到序列学习. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.

[36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. 重新思考计算机视觉的 Inception 架构. *CoRR*, abs/1512.00567, 2015.

[37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. 语法作为外语. In *Advances in Neural Information Processing Systems*, 2015.

[38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 谷歌的神经机器翻译系统：弥合人类与机器翻译的鸿沟. *arXiv preprint arXiv:1609.08144*, 2016.

[39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. 具有快进连接的深度循环模型用于神经机器翻译. *CoRR*, abs/1606.04199, 2016.

[40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. 快速准确的移进-归约成分句法分析. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

注意力可视化

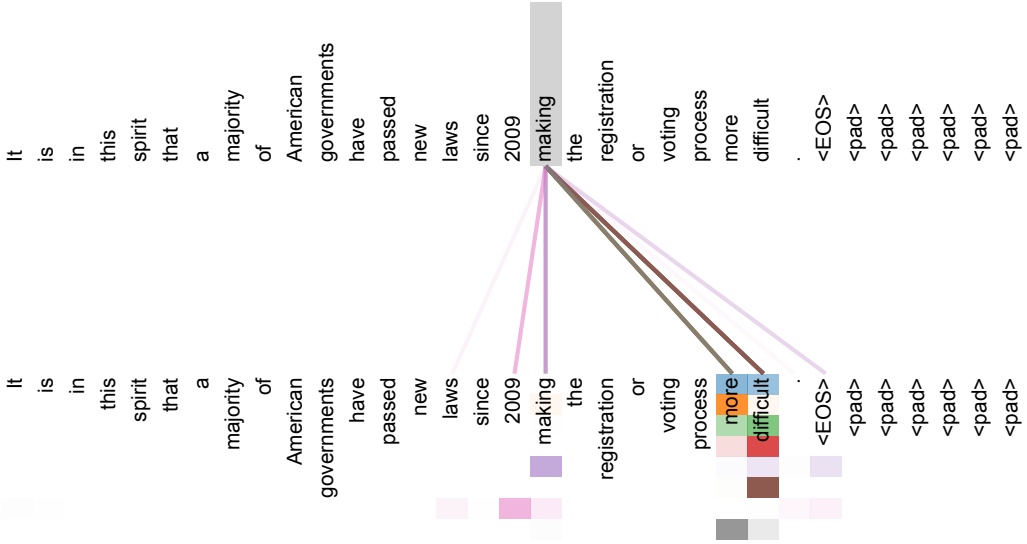

Figure 3: 一个展示编码器自注意力机制（第 5 层，共 6 层）中长距离依赖关系的注意力示例。许多注意力头关注动词”making” 的远距离依赖，完成了短语”making...more difficult”。此处仅展示单词”making” 的注意力。不同颜色代表不同的注意力头。建议彩色查看。

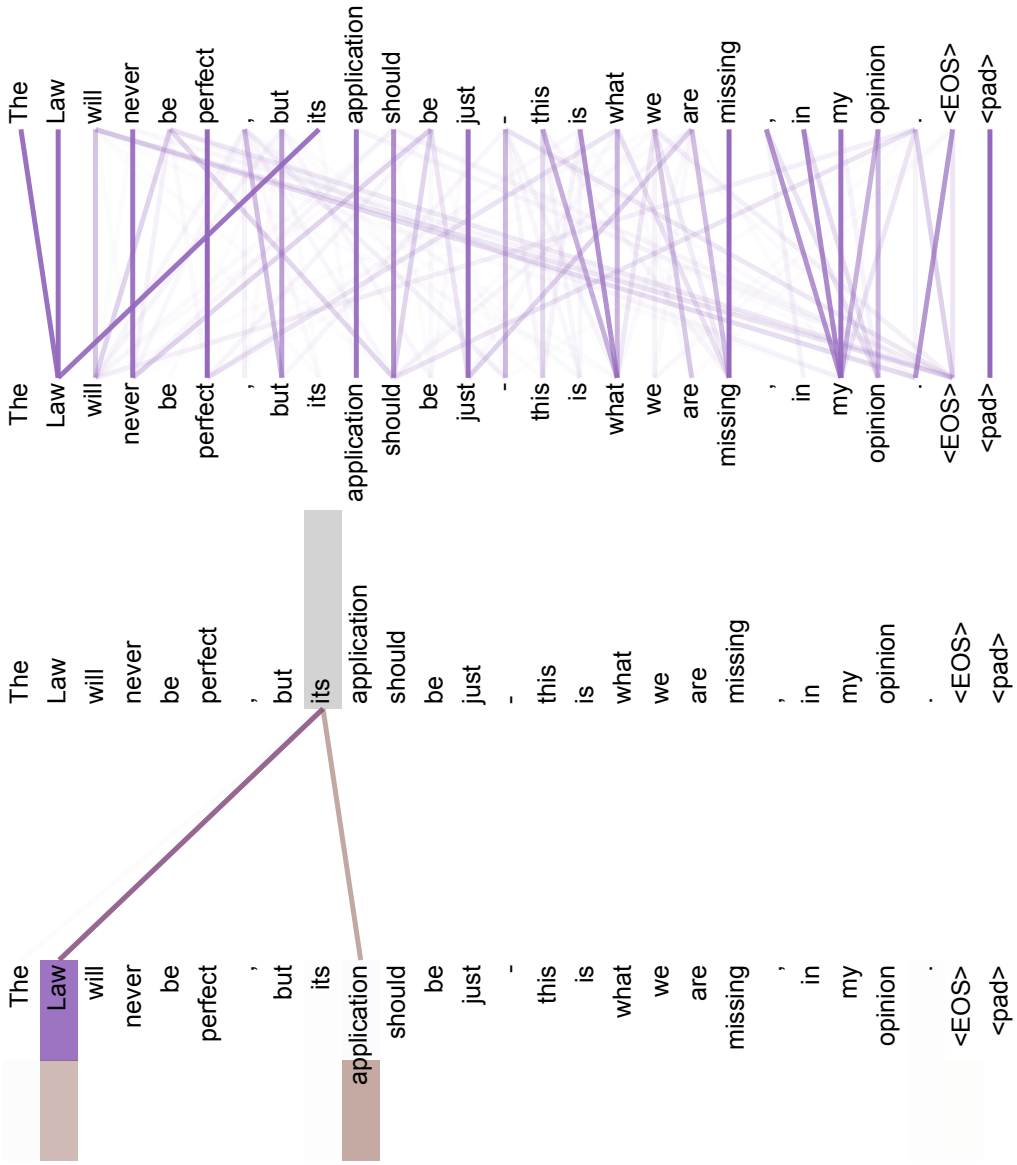

Figure 4: 同样位于第 5 层（共 6 层）的两个注意力头，明显参与了 anaphora resolution（指代消解）。上图：头 5 的完整注意力。下图：仅针对单词"its" 在头 5 和头 6 上的注意力隔离视图。注意该词的注意力分布非常集中。

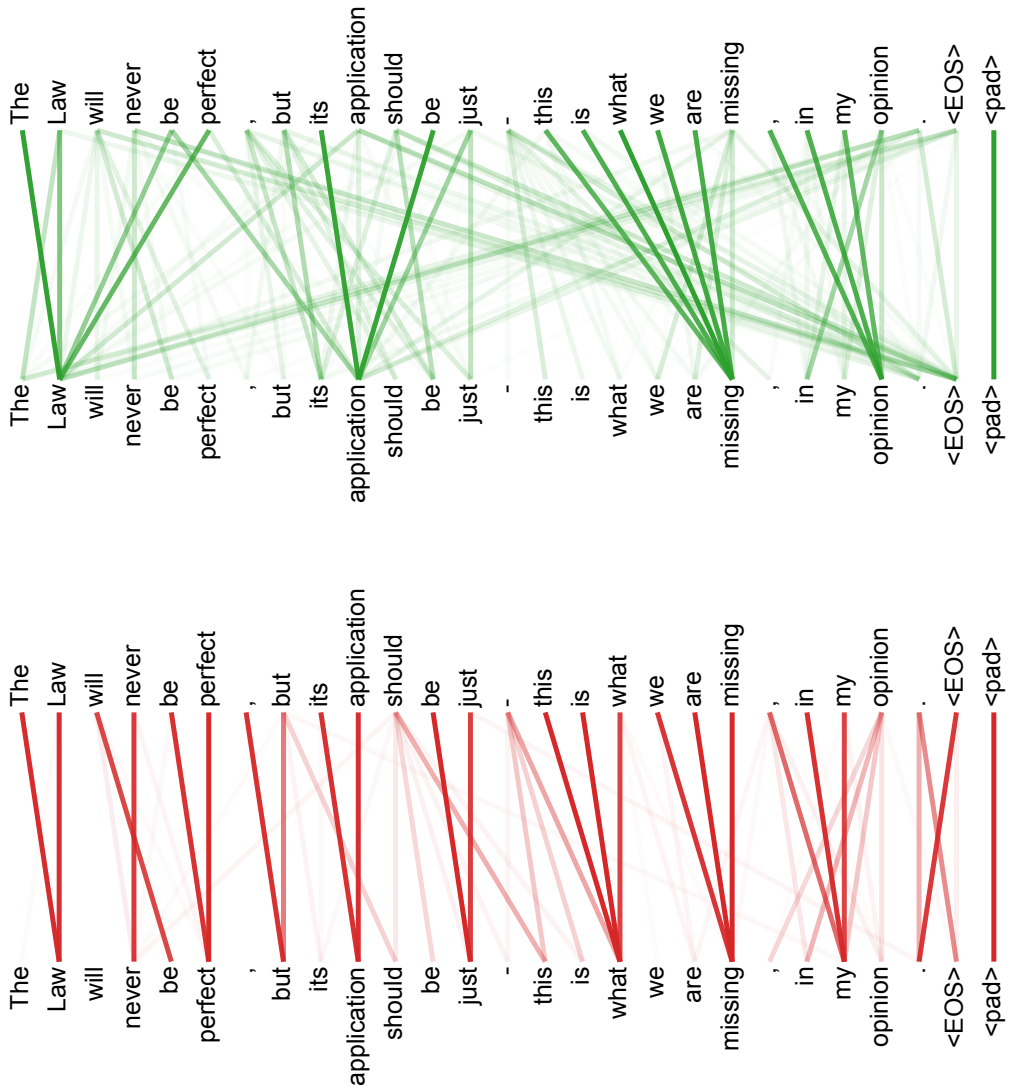

Figure 5: 许多注意力头表现出与句子结构相关的行为模式。我们展示了来自编码器自注意力第 5 层（共 6 层）的两个不同头的两个此类示例。这些注意力头明显学会了执行不同的任务。